

## 实验十二 计算机系统

2023 年秋季学期

*“Next came a very large set of matched volumes containing reference materials: One contained designs for thousands of sleeve bearings, another for computers made of rods, still another for energy storage devices, and all of them were ractive so that she could use them to design such things to her own specifications. Then there were more books on the general principles of putting such things together into systems.”*

– “The Diamond Age: A Young Lady’s Illustrated Primer”, Neal Stephenson

### 12.1 实验目标

本实验的目标是在 Nexys A7-100T 开发板的 FPGA 上实现一个简单的计算机系统，能够运行简单的指令，并处理一定量的输入输出。在所有功能开发完毕后，希望能够完成基本的 `terminal` 功能，即键盘输入命令，并在显示器上输出结果。以下内容只是设计参考，本实验同学们可以自由发挥，完成自己设想的各种计算机功能。具体实现时可以根据自己的兴趣选做一部分或者进行裁剪和修改。

### 12.2 硬件部分

我们在上一个实验已经完成了计算机硬件的核心 CPU 部分。在本实验中，我们将整合之前完成的外设部件，让 CPU 真正能与外设进行交互。这样就可以构成一个真正可以运行的计算机系统。CPU 与外设的通信建议使用内存映射方式。即预先规定好特定的内存地址会对应到特定的外设的输入或输出上去，CPU 用普通的 `load/store` 命令来对这些内存地址进行读写来控制外设。这种方式下 CPU 硬件并不需要关心哪些内存地址是对应外设，哪些是对应数据 RAM。硬件只要将正确的数据地址和读写信号放在总线上，并将总线上的数据正确取回即可。具体数据的含义可以由软件来进行处理。而外设只需要关注自己对应的地址空间的一小块存储即可。

### 12.2.1 外设内存映射

CPU 的数据地址为 32 位，其可寻址空间可以达到 4G Byte。我们前一个实验实现的数据存储只占据了 128K Byte 的空间，因此我们可以将 CPU 的数据寻址空间进行重新规划来同时满足数据存储和外设通信的要求。

首先，可以将高 12 位地址为全零的空间，即 0x00000000 至 0x000fffff 分配给我们的指令存储器。

其次，可以将高 12 位地址为 0x001 的地址空间分配给数据存储器，用于程序正常运行时需要的常量、全局变量、堆及栈空间。当然在我们的系统中指令存储器和数据存储器是分开的，实际上我们可以统一进行编址。

最后，可以分别为每个外设分配特定的空间，以地址高 12 位区分。例如，以 0x002 开头的地址可以分配给显示器，0x003 开头的地址可以分配给键盘，依次类推。

CPU 在读写内存时，会将需要访问的 32 位地址放在数据地址总线上。此时，我们可以根据 CPU 访问地址的高 12 位来判断要使用具体那一片内存。在写入时，只将对应内存的写使能置为有效。在读取的时候，可以从多片不同的存储器中同时读取，最终根据高 12 位地址选择合适的数据放在 CPU 的数据总线上即可。基本思路与实验十中分四片 RAM 来提供 32bit 数据有些类似。

不同的地址区间可以采用不同的方式来实现。例如，数据存储段可以用实验十中的大容量 M10K 存储来实现。对于外设来说，其存储容量一般比较小，所以可以用较为自由的手写 RAM 方式来实现，只要注意读写时钟控制即可。

### 12.2.2 外设内存读写设计

映射到外设的内存一般会需要两个以上的读写端口（即两套地址线），一套供 CPU 使用，一套供外设使用。此时可能会出现读写冲突的情况。所以我们需要对不同的外设对存储空间的读写要求进行分析。典型的外设存储可能包含以下几种类型：

- **CPU 只读型**：此类外设主要包括定时器，或者是板上的开关等等。这类外设对应的空间比较小，一般只有 32 或 64bit。我们只需要将外设对应的 wire 型变量在时钟上升沿赋值给特定寄存器，在 CPU 读取地址匹配时将该寄存器的内容放置在 CPU 的数据总线上即可。
- **CPU 只写型**：此类外设主要是输出设备，例如显示器、LED 或七段显示

等。此时，CPU 总是在上升沿写入数据，外设可以根据自己的逻辑，每次下降沿读取对应的数据放置在自己的寄存器里用于驱动外设。如果有必要，在存储容量小时也可以使用类似寄存器堆的实现方式来非同步读取。

- **CPU 同时需要读写型**：这时同一周期内，CPU 和外设不能同时写入同一地址。此类情况非常少见，一般都可以通过设计将特定空间划分为 CPU 只读或只写区域。具体设计可以参考后面的键盘空间设计。



C 语言中有一个重要的关键字 **volatile** 用于防止编译器对代码进行过度编译优化。例如，对于 CPU 只读型变量，对应地址的内存有可能会被外设改变。CPU 对外设轮询时可能是使用 `while(data==0) {...}` 这样的循环来等待外设输入的。但是，“智能”的编译器在打开 O2 之类优化的选项时会试图优化这段代码。当循环内部没有改变 **data** 这个变量时，编译器可能认为这个循环是没有用的，只需要做一次判断。这样，“优化”后的二进制代码就和原始代码的本意不一致了。在这种情况下，需要在 **data** 这个变量的申明中增加 **volatile** 修饰，说明该变量有可能被外界改变，避免编译优化出问题。

### 12.2.3 常见外设实现

下面简单介绍一下板上常见外设的实现方式。

**LED**: CPU 只负责写入，可以只用 32bit 单个寄存器实现。将此寄存器的对应比特直接 **assign** 给 LED 即可。CPU 软件在自身的存储空间保留一份 LED 的当前状态，在需要改变 LED 亮暗时可以对自身的副本进行操作，然后直接将副本 SW 至 LED 对应的地址。

**七段数码管**: CPU 只负责写入七段数码管的 BCD 码，同 LED 类似，也可以直接用 32bit 寄存器实现。

**定时器**: CPU 只读型。如果需要提供毫秒或微秒量级的定时器，可以用系统时钟分频实现 1 毫秒或 1 微秒的定时信号。通过计数器累加，并在每个时钟的上升沿将数据写入对应的定时器内存空间中。定时器宽度可以是 32 位或 64 位的。CPU 在需要访问定时器时可以直接用 **load** 指令读取对应定时器数据，这样就可以获取从开机至当前经过的毫秒或微秒数。该功能可以用于实现时钟、计算程序运行时间，有兴趣的同学还可以考虑用最小堆或时间轮的方法来实现操作系统中的各种定时功能。

**开关：**CPU 只读型，只需要将开关连到特定寄存器即可通过 Load 指令读取对应的开关状态。

**显示器：**CPU 只写型。可以为显示器分配一定量的字符显存，每 8bit 对应一个 ASCII 码。例如要支持 64 行 × 64 列的字符缓存只需要 4096Byte 即可。此类存储需要支持 CPU 不同位宽的 sb,sh,sw 指令。除此之外，可以再单独分配一些控制寄存器对应的内存空间。例如，可以分配一个起始行号寄存器，方便实现滚屏操作；也可以分配一个颜色控制寄存器来控制字符和背景颜色。CPU 在自身时钟的上升沿写入显存和显示控制寄存器。而显示器部分类似我们实验九中的功能，只需要负责从显存中读取 ASCII 码，并且正确输出即可。为实现滚屏等功能，显示器硬件可以从给定的一个起始行号开始读取 ASCII 码，随后显示起始行号后 30 行的内容（行号可以是模 64 循环的）。这样就可以通过改变起始行号方便地实现滚屏。注意，建议显示器只忠实地显示显存中的 ASCII 字符，滚屏清零等逻辑由 CPU 软件实现。显示器读取显存可以用自己的 25MHz 时钟，如果 CPU 无法达到这个频率对于实现上影响也不是很大。即 CPU 写入可以比较慢，显示器仍然可以按自己的步伐来读取显存。



图 12-1: 显存组织方式示例

**键盘：**键盘建议使用循环缓冲区的方式来实现。我们首先分配可以存放 16 或 32 个扫描码（4 字节）的内存空间。同时，分别设置头指针 head 和尾指针 tail。此时，键盘硬件作为数据生成者负责写入缓冲区。而 CPU 是数据消费者，负责从缓冲区中读取数据。在这个数据结构中，头指针只有 CPU 会写入，

尾指针和缓冲区只有键盘会写入，因此可以将 CPU 只写和外设只写的区域分开，不会读写冲突。键盘在每次收到一个新的按键时，读取 head 和 tail 两个指针，如果  $tail=head-1$ ，说明缓冲区已满，不能写入。否则，键盘就在 tail 处写入按键对应的扫描码，然后将  $tail+1$ 。对于 CPU 来说，每次需要检查有无键盘输入时，可以首先读取 head 和 tail，如果  $head==tail$ ，说明缓冲区为空，没有键盘输入，CPU 可以直接返回继续其他工作。如果 head 和 tail 不相等，CPU 将 head 指针指向的扫描码拷贝入自己的内存中，再将  $head+1$ ，表明已经读取了该扫描码。这样，我们可以用缓冲区记录一部分按键，让 CPU 在忙的时候不会丢失按键，同时也可以实现非阻塞式读取按键。CPU 读取扫描码后可以用软件对通码和断码进行处理，并进行扫描码到 ASCII 的转换。具体软件实现可以自行设计。如果系统只有对 ASCII 码的读取需求，也可以在缓冲区中直接放置 ASCII 码，即扫描码到 ASCII 码的转换由硬件完成。此方案比软件转换的方案灵活性要差一些。

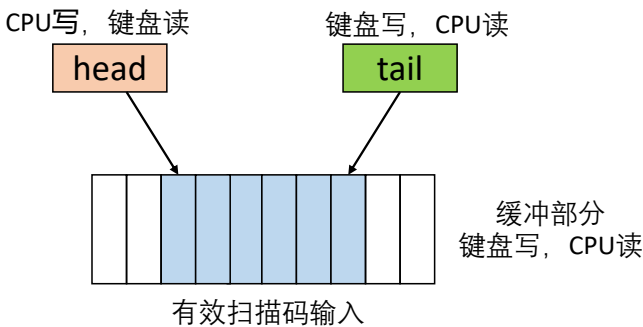


图 12-2: 键盘缓冲区组织方式示例

12.2.4 内存映射的具体实现

本实验设计中的数据 RAM 都是以双口方式实现的，即 RAM 提供单独的读地址、读数据输出口、读时钟，以及写地址、写数据输入口和写时钟。这种情况下 RAM 的读和写操作是完全分离的，可以分别单独进行设计。我们可以用单独的 RAM 或者寄存器来实现不同外设的数据存储空间。针对不同的外设的读写要求，将外设的数据线、地址线及 CPU 的数据线、地址线分别连接到对应外设的数据模块的读/写接口上即可。以数据存储为例，数据存储只需要与 CPU 相连，所以我们将读写地址、时钟和 memop 这些信号均直接与 CPU 相连。

```
1      dmem datamem(.rdaddr(drdaddr),      //CPU 读地址
```

```

2         .dataout(ddataout), // 数据存储读数据输出
3         .wraddr(dwraddr),   //CPU写地址
4         .datain(ddatain),   //CPU写数据输入
5         .rdclk(drdclk),     //CPU读时钟
6         .wrclk(dwrclk),     //CPU写时钟
7         .memop(dop),        //CPU数据操作类型
8         .we(datawe));      // 数据存储写使能。

```

为了实现地址空间的分配，我们需要在 CPU 读写以 0x001 开头的地址时才对数据存储进行操作。因此，数据存储中有两个信号没有直接和 CPU 直连。对于写操作，只需要控制写使能就能确定是否需要当前存储进行写操作，所以可以先比较地址再确定是否写使能：

```
1 assign datawe=(dwraddr[31:20]==12'h001)? dwe:1'b0;
```

这句话判断写地址高 12 位是否为 0x001，如果是则写使能按照 CPU 输出的写使能 dwe 设置，否则总是不写入。对于读操作，我们可以将各个内存块读取的数据根据 CPU 读地址的高位来进行选择：

```
1 assign ddata=(drdaddr[31:20]==12'h001)? ddataout:
2         ((drdaddr[31:20]==12'h003)? keymemout :32'b0 );
```

这里在地址高位为 0x001 时选择了数据存储的输出，为 0x003 时选择了键盘的输出。显示器由于 CPU 不用读取，所以没有设置。

## 12.3 软件部分

在实现了 CPU 及对外设的内存映射后，我们可以开始编写软件系统。我们建议大家使用 RISC-V 的 C 语言工具链对软件进行编译，这样可以使用 C 语言来编写整体代码。

### 12.3.1 编译过程介绍

我们需要利用上个实验中安装过的 riscv32-unknown-elf 工具链来编译系统软件。请自行编写合适的 Makefile 进行编译，我们提供了一个简单的 makefile 示例如下：

```

1 default: all
2

```

```

3  XLEN ?= 32
4  RISCVP_PREFIX ?= riscv$(XLEN)-unknown-elf-
5  RISCVP_GCC ?= $(RISCVP_PREFIX)gcc
6  GCC_WARNINGS := -Wall -Wextra -Wconversion -pedantic -Wcast-qual -Wcast-align
7      -Wwrite-strings
8  RISCVP_GCC_OPTS ?= -static -mcmodel=medany -fvisibility=hidden -Tsections.ld
9      -nostdlib -nolibc -nostartfiles ${GCC_WARNINGS}
10 RISCVP_OBJDUMP ?= $(RISCVP_PREFIX)objdump --disassemble-all --disassemble-zeroes
11     --section=.text --section=.text.startup --section=.text.init
12     --section=.data
13 RISCVP_OBJCOP ?= $(RISCVP_PREFIX)objcopy -O verilog
14 RISCVP_HEXGEN ?= 'BEGIN{output=0;}{ gsub("\r","",$(NF)); if ($$1 ~/@/)
15     {if ($$1 ~/@00000000/) {output=code;} else {output=1- code;}};
16     gsub("@","0x",$$1); addr=strtonum($$1); if (output==1)
17     {printf "%08x\n", (addr%262144)/4;}}
18     else {if (output==1) { for(i=1;i<NF;i+=4)
19     print $(i+3)$(i+2)$(i+1)$$i;}}}'
20 RISCVP_COEGEN ?= 'BEGIN{printf "memory_initialization_radix=16;\n
21     memory_initialization_vector=\n"; addr=0;} { gsub("\r","",$(NF));
22     if ($$1 ~/@/) {gsub("@","0x",$$1);temp=strtonum($$1);
23     for(;addr<temp;addr++){print "00000000,";}} else {printf "%s,\n",
24     $$1; addr=addr+1;}} END{print "0;\n";}'
25
26
27 SRCS := $(wildcard *.c)
28 OBJS := $(SRCS:.c=.o)
29 EXEC := main
30
31 .c.o:
32     $(RISCVP_GCC) -c $(RISCVP_GCC_OPTS) $< -o $@
33
34 ${EXEC}.elf : $(OBJS)
35     ${RISCVP_GCC} ${RISCVP_GCC_OPTS} -e entry $(OBJS) -o $@
36     ${RISCVP_OBJDUMP} ${EXEC}.elf > ${EXEC}.dump
37
38 ${EXEC}.tmp: ${EXEC}.elf
39     $(RISCVP_OBJCOP) $< $@

```

```
40
41 ${EXEC}.hex: ${EXEC}.tmp
42     awk -v code=1 $(RISCV_HEXGEN) $< > $@
43     awk -v code=0 $(RISCV_HEXGEN) $< > ${EXEC}_d.hex
44
45 ${EXEC}.coe: ${EXEC}.hex
46     awk ${RISCV_COEGEN} $< > $@
47     awk ${RISCV_COEGEN} ${EXEC}_d.hex > ${EXEC}_d.coe
48
49 .PHONY: all clean
50
51 all: ${EXEC}.coe
52
53 clean:
54     rm -f *.o
55     rm -f *.dump
56     rm -f *.tmp
57     rm -f *.elf
58     rm -f *.hex
59     rm -f *.coe
```

Makefile 所定义的内容主要包括：

- 第 3-6 行，定义工具链编译命令及 gcc 的告警级别。
- 第 8-9 行，定义 gcc 链接的参数，这里注意使用了静态链接，并且关闭了所有标准库的链接。除此之外，我们还使用了 sections.ld 来对二进制文件的各个段地址进行了规定，具体参见后面对 sections.ld 文件的解释。
- 第 10-13 行，类似上个实验中的内容，定义 OBJDUMP 和 OBJCOPY 的参数
- 第 14-19 行，利用 awk 对输出的十六进制文本文件进行转换，生成 verilog 可以读取的 4 字节 hex 文件。同时将代码段和数据段分开。
- 第 20-24 行，利用 awk 对 hex 文件进行改写，生成 Vivado 支持的 coe 文件分别初始化指令存储器和数据存储器。注意，在生成 IP 核中的 BRAM 时，可以指定使用 coe 文件进行初始化，但是每次 coe 文件更改后需要重新生成 IP 核，在打包 IP 核的时候 Vivado 会将 coe 文件转换为 mif 文件，编译的时候用 mif 初始化内存。所以感兴趣的同学也可以考虑直接生成 mif 文件 hack 进 IP 核存储初始化文件。



- 第 26-35 行，扫描目录下所有.c 文件，预备生成所有对应的.o 文件，最后用所有的.o 文件来链接生成 main.elf 二进制执行文件。
- 第 37-46 行，在生成 main.elf 之后，将二进制文件分多步转换成对应的 main.coe 和 main\_d.coe 来初始化指令存储器和数据存储器。
- 第 48-58 行，定义 make 的基本操作，make clean 清除所有输出文件。

同学们可以自行学习和了解 Makefile 的编写规则，按实际工程的需求对 Makefile 进行改写。

在 Make 过程中，我们利用 sections.ld 文件来规定可执行文件的地址映射，该文件具体的示例如下：

```
1 ENTRY(entry)
2 OUTPUT_FORMAT("elf32-littleriscv")
3
4 SECTIONS {
5     . = 0x00000000;
6     .text : {
7         *(entry)
8         main.o (.text)
9         *(.text*)
10        *(text_end)
11    }
12    etext = .;
13    _etext = .;
14    . = 0x00100000;
15    .rodata : {
16        *(.rodata*)
17    }
18    .data : {
19        *(.data)
20    }
21    edata = .;
22    _data = .;
23    .bss : {
24        _bss_start = .;
25        *(.bss*)
26        *(.sbss*)
```

```
27         *(.scommon)
28     }
29     _stack_top = ALIGN(1024);
30     . = _stack_top + 1024;
31     _stack_pointer = .;
32     end = .;
33     _end = .;
34     _heap_start = ALIGN(1024);
35 }
```

该文件主要规定了二进制可执行文件的地址分配。首先，第 1 行说明了程序入口为 `entry` 函数（事实上在我们的系统里可以不用规定）。从第 5 行开始，我们规定了代码和数据的排布方式。最重要的是 `.text` 代码段的规定。在硬件中我们规定了 `reset` 后从 `0x00000000` 地址开始执行，所以需要规定代码段从 `0x00000000` 开始，同时我们要把我们的入口函数 `entry` 放在代码段的开始位置，这是在第 7 行中规定的。后续的函数顺序可以自行定义。

在第 14 行规定了数据段的开始地址是 `0x00100000`，这我们的硬件规定是一致的。在 `MakeFile` 里，我们也会专门利用 `awk` 提取数据段内容（默认非代码段的数据全部放入数据存储），统一放入 `main_d.mif` 中用于初始化数据存储。

对于内存映射的输出输出地址空间我们不需要进行初始化，所以在二进制可执行文件中没有体现。

## 12.3.2 Hello World 示例

我们提供了简单的 Hello World 代码供大家参考，其中的 `main.c` 内容如下：

```
1  #include "sys.h"
2
3  char hello[]="Hello World!\n";
4
5  int main();
6
7  //setup the entry point
8  void entry()
9  {
10     asm("lui sp, 0x00120"); //set stack to high address of the dmem
```

```

11     asm("addi sp, sp, -4");
12     main();
13 }
14
15 int main()
16 {
17     vga_init();
18    _putstr(hello);
19     while (1)
20     {
21     };
22     return 0;
23 }

```

main 文件首先包含了本系统自定义的头文件。第 3 行中用全局变量保存了 Hello World 字符串，该数据在编译后会放在数据段内。

**程序入口：**在代码的第 7-13 行，我们定义了一个入口 entry 函数。该函数主要作用是初始化系统堆栈，并调用 main 函数。在链接过程中我们通过 sections.ld 来将该函数置于起始 0x00000000 地址。使用 entry 函数的主要目的是初始化堆栈指针，在操作系统调用 main 函数前是会初始化 sp 的。但是我们的裸机程序中 sp 启动时总是全零，如果不初始化，main 函数第一句调整 sp 就会将 sp 变为 0xffffffff，访问到我们地址空间不存在的部分。

```

1 00000030 <main>:
2 30: ff010113      addi    sp,sp,-16
3 34: 00112623      sw     ra,12(sp)
4 38: 00812423      sw     s0,8(sp)
5 3c: 01010413      addi    s0,sp,16
6 40: 1f8000ef      jal    ra,238 <vga_init>
7 44: 00100517      auipc   a0,0x100
8 48: fbc50513      addi    a0,a0,-68 # 100000 <hello>
9 4c: 324000ef      jal    ra,370 <_putstr>
10 50: 0000006f      j      50 <main+0x20>

```

而 entry 函数强制通过汇编重置了 sp，使其位于我们数据段的顶端 0x0011fffc 处，这样就保证了代码运行过程中堆栈是可以正常访问的。当然，当 main 函数返回 entry 之后系统状态会不正常，所以要求 main 函数不返

回，在内部通过死循环 Halt。

```

1 00000000 <entry>:
2   0:   ff010113          addi    sp,sp,-16
3   4:   00112623          sw      ra,12(sp)
4   8:   00812423          sw      s0,8(sp)
5   c:   01010413          addi    s0,sp,16
6  10:   00120137          lui     sp,0x120
7  14:   ffc10113          addi    sp,sp,-4 # 11fffc <_end+0x1
      f7fc>
8  18:   018000ef          jal     ra,30 <main>
9  1c:   00000013          nop
10 20:   00c12083          lw      ra,12(sp)
11 24:   00812403          lw      s0,8(sp)
12 28:   01010113          addi    sp,sp,16
13 2c:   00008067          ret

```

main 函数执行的内容比较简单，主要是初始化 VGA 缓存，并且调用库函数输出 Hello World，完成后进入死循环。

在 sys.h 头文件里，我们规定了 VGA 缓存的起始地址，VGA 行列数量等基本常数和部分库函数。同学们可以自己编写输入输出库函数或者移植 PA 中的部分代码。

```

1 #define VGA_START      0x00200000
2 #define VGA_MAXLINE    30
3 #define VGA_MAXCOL     70
4
5
6 void putstr(char* str);
7 void putch(char ch);
8
9 void vga_init(void);

```

在 sys.c 中，我们实现了部分输出的库函数，包括 putch, putstr 等等。

```

1 #include "sys.h"
2
3 char* vga_start = (char*) VGA_START;
4 int    vga_line=0;
5 int    vga_ch=0;

```

```
6
7 void vga_init(){
8     vga_line = 0;
9     vga_ch =0;
10    for(int i=0;i<VGA_MAXLINE;i++)
11        for(int j=0;j<VGA_MAXCOL;j++)
12            vga_start[ (i<<7)+j ] =0;
13 }
14
15 void putch(char ch) {
16     if(ch==8) //backspace
17     {
18         //TODO
19         return;
20     }
21     if(ch==10) //enter
22     {
23         //TODO
24         return;
25     }
26     vga_start[ (vga_line<<7)+vga_ch] = ch;
27     vga_ch++;
28     if(vga_ch>=VGA_MAXCOL)
29     {
30         //TODO
31     }
32     return;
33 }
34
35 void putstr(char *str){
36     for(char* p=str;*p!=0;p++)
37         putch(*p);
38 }
```

如果同学们需要实时对数据段进行更新，建议可以修改硬件，增加一个只读的数据存储，在 main 函数起始时通过 memcpy 把只读的存储拷贝到实际的数据存储器中去。

### 12.3.3 未实现指令的执行

RV32I 指令中没有常用的乘法指令，但是 gcc 在编译过程中如果遇到整数乘法会直接调用软件乘法函数 `__mulsi3` 来通过软件完成整数乘法操作。如果在代码中需要用到整数乘除法操作，可以参考 gcc 中的 `__mulsi3` 实现 (<https://github.com/gcc-mirror/gcc/blob/master/libgcc/config/epiphany/mulsi3.c>)，将对应的函数链接到主程序中即可。

```
1 unsigned int __mulsi3(unsigned int a, unsigned int b) {
2     unsigned int res = 0;
3     while (a) {
4         if (a & 1) res += b;
5         a >>= 1;
6         b <<= 1;
7     }
8     return res;
9 }
10
11 unsigned int __umodsi3(unsigned int a, unsigned int b) {
12     unsigned int bit = 1;
13     unsigned int res = 0;
14
15     while (b < a && bit && !(b & (1UL << 31))) {
16         b <<= 1;
17         bit <<= 1;
18     }
19     while (bit) {
20         if (a >= b) {
21             a -= b;
22             res |= bit;
23         }
24         bit >>= 1;
25         b >>= 1;
26     }
27     return a;
28 }
29
```

```
30 unsigned int __udivsi3(unsigned int a, unsigned int b) {
31     unsigned int bit = 1;
32     unsigned int res = 0;
33
34     while (b < a && bit && !(b & (1UL << 31))) {
35         b <<= 1;
36         bit <<= 1;
37     }
38     while (bit) {
39         if (a >= b) {
40             a -= b;
41             res |= bit;
42         }
43         bit >>= 1;
44         b >>= 1;
45     }
46     return res;
47 }
```

## 12.4 调试工具

随着实验规模的扩大，同学们可能会发现编译及调试越来越耗费时间。因此，为了更好地完成综合实验，需要合理地开发调试工具。在综合实验中，每次硬件编译可能会耗费十几分钟到半小时不等，而且软件修改往往也需要重新编译。除此之外，硬件故障由于内部信号的不可见的特点，往往花费时间修改并编译后还不一定能迅速寻找到故障点。在本节中，我们介绍通过串口来实现软件下载，CPU 重启等功能，并提供了示例代码。同学们可以根据自己的工程需求对工具进行改造，以更加高效地完成综合实验。

### 12.4.1 串口背景知识

串行接口 (serial port) 将数据逐个比特地在单条线路上进行传输。这种传输模式由于使用到的数据线数量较少，因而在计算机和数字设备中得到广泛应用。由于 FPGA 及嵌入式设备一般缺乏简便的输入输出机制，工程师们往往选择使用串口来对设备进行调试和设置。所以，在系统中实现一个简单的串口控

制器往往能够大大减轻开发负担。

在旧一些的台式机上往往包含 9 针 (DB9 或 DE9) 的 RS232 串行接口。如图 12-3 所示：

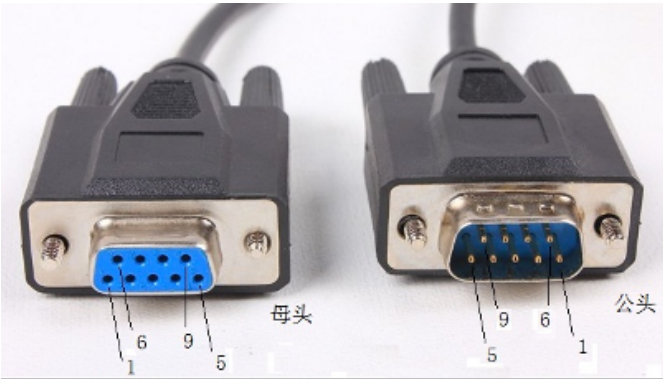


图 12-3: DB9 RS232 串口接头

DB9 接口的引脚如下表，其中数据主要通过 2、3 引脚和地线传输。在接线时需要将收端 2 号引脚接到发端 3 号引脚，并将发端 2 号引脚接到收端 3 号引脚，形成双向数据链路。串口传输一般支持全双工 (Full Duplex) 通信，即收发两端可以同时收发数据。

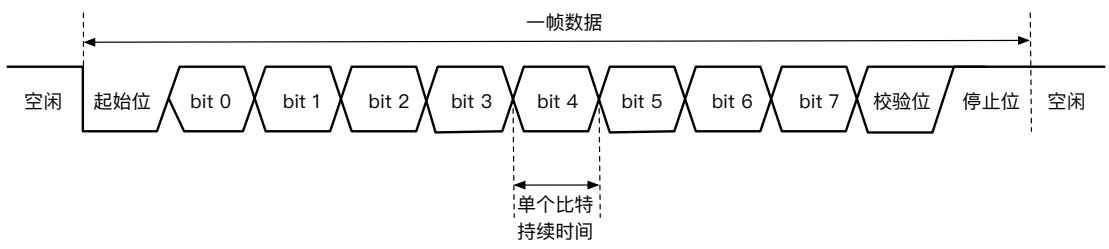
表 12-1: RS232 引脚说明

| 引脚号 | 功能      | 缩写  |
|-----|---------|-----|
| 1   | 数据载波检测  | RCD |
| 2   | 接收数据    | RXD |
| 3   | 发送数据    | TXD |
| 4   | 数据终端准备  | DTR |
| 5   | 信号地     | GND |
| 6   | 数据设备准备好 | DSR |
| 7   | 请求发送    | RTS |
| 8   | 可以发送    | CTS |
| 9   | 振铃指示    | RI  |

现代计算机一般已用 USB 取代了串口。本实验主要针对通用异步收发传输器 (Universal Asynchronous Receiver/Transmitter, UART) 来进行设计。UART 采用异步串行方式通信，收发两端不需要时钟同步。在异步条件下，接收端需



要自己产生时钟来读取数据。典型的串口通信帧结构如图 12-4所示：



类似于我们之前用过的 PS/2 接口，每一帧的起始也是通过高电平变为低电平来指示，后续可以跟随 8 位数据位、一位校验位和一位高电平的停止位。由于是异步通信，接收端需要自己产生时钟。异步串行通信的时钟是由通信速率来决定的，一般我们称其为波特率，也就是每秒钟传输的 bit 数。常用的串口通信波特率包括 9600bps (bit per second)，115200bps 等等。对于 9600bps，每个 bit 的宽度就是  $1/9600=0.00010417$  秒 = 0.10417 毫秒。除了波特率，串口通信一般还可以设置各种通信参数，包括数据位的位数、是否有校验位、停止位的长度等等。这些参数需要通信双方在通信前预先约定好。常见的参数选择如下表所示。

表 12-2: 常见串口参数

| 参数        | 常见选择范围                            |
|-----------|-----------------------------------|
| 波特率       | 9600, 19200, 38400, 57600, 115200 |
| 数据位 bit 数 | 7, 8                              |
| 校验位       | 无校验位，奇校验，偶校验                      |
| 停止位       | 1 位，1.5 位，2 位                     |

我们建议大家可以选择 **115200bps，8 位数据，奇校验及 1 位停止位**。同学们也可以选择使用其他设置或动态配置。

**FPGA 开发板串口：**在我们使用的 Nexys A7-100T 开发板中，串口是通过 USB-UART 桥接芯片 FT2232 来实现的。串口和我们用于 FPGA 程序下载的 USB 接口共用一根线，但不会相互干扰。也就是说下载完 bitstream 后不用拔线，FPGA 可以直接和电脑进行串口通信。串口的 TXD、RXD 及 CTS、RTS 分别连接 FPGA 的 C4、D4、D3 和 E5 接口。注意，图 12-5中的 TXD、RXD、

CTS 和 RTS 是以电脑接口标注的，输入输出方向请按箭头指示，切勿弄错收发方向。其中 CTS、RTS 用于硬件流控，可以不用处理，只需要将 CTS 置低即可。

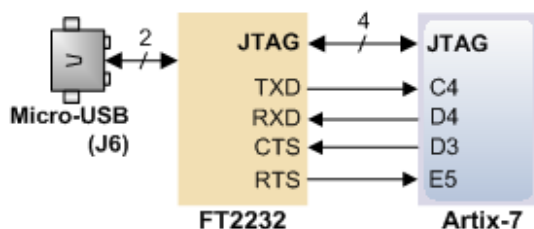


图 12-5: Nexys A7-100T 串口连接

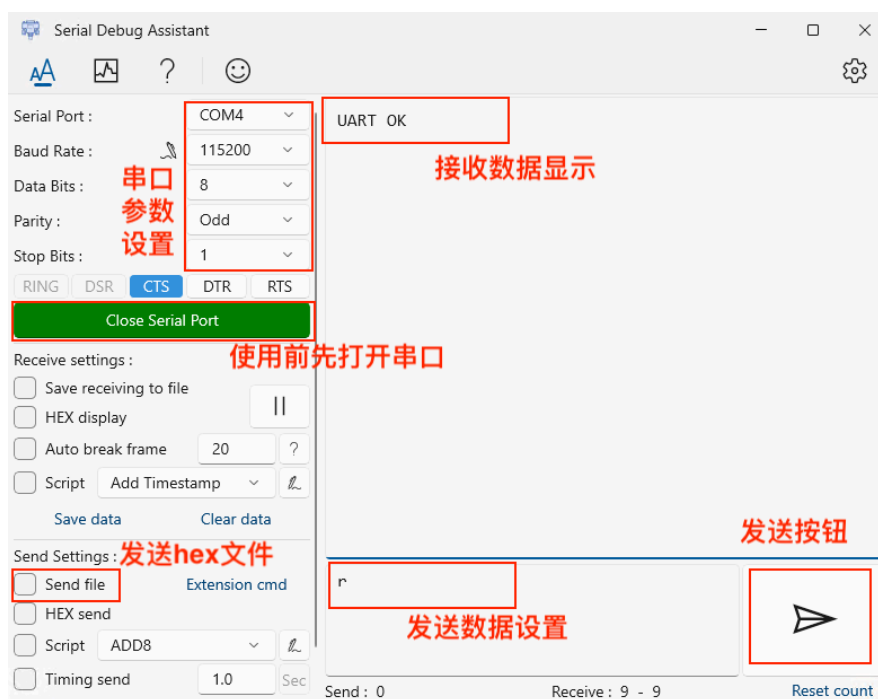


图 12-6: 串口调试工具界面

**电脑侧串口：**开发板与电脑的连接直接通过 USB 线，调试时只需要安装软件即可。同学们可以自行安装串口调试工具，例如 Windows App 商店里提供的 Serial Debug Assistant。在连接 USB 后，会看到 USB 转换的串口（一般是 COM4），通过该串口即可与 Nexys A7-100T 开发板进行通信。Nexys A7-100T 开发板在 USB 接口附近有 2 个黄色的 LED，分别指示当前串口是否在收发。当串口调试工具打开串口，并开始通信时，收发 LED 会随每次收发闪烁，可以

帮助确定收发是否正常。通过串口调试工具，可以向 FPGA 收发文本命令（注意，串口缺省是不回显的，即发出去的命令不一定会显示在接收窗口），或者直接发送内存初始化文件.hex 来直接更新指令或数据存储器。

## 12.4.2 串口收发设计

为简化实验，我们提供了 Nexys A7-100T 串口收发代码以及相关的调试基础功能。感兴趣的同学可以在示例代码上增加自己需要的功能。为了帮助大家理解串口代码，我们下面对串口代码进行适当解释。

**串口发送部分：**串口发送的基本原理是将数据一位一位的按给定波特率发送到对端的 RXD 端口上去。发送端模块的代码如下，其中输入时钟 `clk` 是 100MHz，用于产生 115200Hz 的发送时钟。八比特的数据输入 `data` 通过串行方式由送至 `tx_out`，最终该 `tx_out` 信号会连接到顶层模块中的 `UART_RXD_OUT` 信号上去。除此之外，模块还提供了输出 `tx_ready` 和输入 `tx_enable` 两个握手信号，用于和上层模块通信。

```
1 module serial_tx( input clk,
2                   input [7:0] data,
3                   input tx_enable,
4                   output reg tx_out,
5                   output tx_ready );
6   wire tx_clk; // serial clock
7   reg tx_busy;
8   reg [10:0] tx_buf;
9   reg [3:0] tx_count;
10  initial
11  begin
12      tx_count= 4'd0;
13      tx_busy = 1'b0;
14      tx_out = 1'b1;
15  end
16  clkgen #(115200) mytxclk(clk,1'b0,1'b1,tx_clk); //always enable tx_clock
17
18  assign tx_ready = ~ tx_busy;
19  always@(posedge tx_clk) //state change only at the tx_clk edge
20  begin
```

```
21     if(tx_busy) //busy, ignore tx_enable and send
22     begin
23         tx_out<=tx_buf[0];
24         tx_buf[9:0]<=tx_buf[10:1]; //shift tx_buf
25
26         if(tx_count>=4'd10) //finished
27         begin
28             tx_busy = 1'b0;
29             tx_count<=4'd15;
30         end
31     else
32     begin
33         tx_busy = 1'b1;
34         tx_count<=tx_count+4'd1;
35     end
36 end
37 else
38 begin
39     tx_out=1'd1;
40     if(tx_enable) //load the buffer and start next cycle
41     begin
42         tx_buf[0]<=1'b0;
43         tx_buf[8:1]<=data[7:0];
44         tx_buf[9]<=~(^data[7:0]);
45         tx_buf[10]<=1'b1;
46         tx_busy<=1'd1;
47         tx_count<=4'd0;
48     end
49     else //no data
50     begin
51         tx_buf<=tx_buf;
52         tx_busy<=1'd0;
53         tx_count<=4'd0;
54     end
55 end
56 end
57 endmodule
```

发送端的逻辑相对简单，在上层模块需要发送时，将数据 `data` 准备好，然后将 `tx_enbale` 置 1，此时发送模块将起始位、数据、校验位和停止位准备好，然后发送端变为忙 `tx_busy`，开始按位发送。发送完毕后将 `tx_ready` 置 1，准备下一次发送。

**串口接收部分：** 串口接收端类似 PS/2 接口，先对信号进行同步，寻找信号的沿，然后产生和发送端一样的时钟，并在时钟周期的中间来读取串口数据。这里和 PS/2 不同的是，串口只有一根数据线，没有时钟，时钟需要接收端自己生成。

```
1 module serial_rx(input clk,
2                 input txd_in,
3                 output reg [7:0] rx_data,
4                 output reg rx_finish,
5                 input rx_ready);
6
7 wire rx_clk;
8 reg [2:0] clk_sync;
9 reg [2:0] start_sync;
10 reg [3:0] rx_count;
11 reg [10:0] rx_buf;
12 wire sampling;
13 wire start_bit;
14 reg rx_busy;
15 reg rx_clk_rst;
16
17 initial
18 begin
19     rx_finish = 1'b0;
20     rx_busy = 1'b0;
21     rx_clk_rst = 1'b0;
22 end
23
24 clkgen #(115200) mytxclk(clk,rx_clk_rst,rx_busy,rx_clk); //rst clk at negedge
25
26 always @ (posedge clk)
27 begin
```

```
28     clk_sync <= {clk_sync[1:0],rx_clk};
29 end
30 assign sampling = ~clk_sync[2] & clk_sync[1]; //sampling of receive clock
31 // posedge because we start the clock at begining of the bit
32
33 always @ (posedge clk)
34 begin
35     start_sync <= {start_sync[1:0],txd_in};
36 end
37 assign start_bit = start_sync[2] & ~start_sync[1]; //find negedge of start bit
38 always @ (posedge clk) //triger the clock when find negedge
39 begin
40     if(rx_busy) //receiving
41     begin
42         rx_clk_rst<=1'b0; //reset only last one clock
43         if(rx_finish)
44         begin
45             rx_busy<=1'b0;
46         end
47     else
48     begin
49         rx_busy<=rx_busy;
50         if(sampling)
51         begin
52             if(rx_count>=4'd10)
53             begin
54                 if( (rx_buf[0]==0)&&(^rx_buf[9:1]) )
55                 begin
56                     rx_data[7:0]<=rx_buf[8:1];
57                     rx_finish<=1'b1;
58                 end
59             end
60         else
61         begin
62             rx_buf[rx_count]<= txd_in;
63             rx_count <= rx_count + 4'd1;
64         end
65     end
66 end
```

```
65         end
66     end
67 end
68 else//not busy
69 begin
70     if(start_bit & (~rx_finish)) //start receiving when find start bit
71     begin
72         rx_busy<=1'b1;
73         rx_clk_rst<=1'b1;
74         rx_count<=4'd0;
75         rx_data<=8'd0;
76         rx_finish<=1'b0;
77     end
78 else
79     begin
80         rx_busy<=1'b0;
81         rx_clk_rst<=1'b0;
82     end
83     if(rx_ready)
84     begin
85         rx_finish<=1'b0;
86     end
87 end
88 end
89 endmodule
```

接收端的接口也包括了 100MHz 的时钟输入 `clk` 用于产生串口时钟。此处时钟注意要在收到第一个开始位的下降沿时 `reset`，保持与发送端同步。串口先通过 `txd_in` 信号接入，该信号最终会接上顶层模块的 `UART_TXD_IN` 端口。顶层模块串口的另外两根信号线 `UART_CTS` 和 `UART_RTS` 在实际使用中可以缺省处理，及对于输出的 `UART_CTS` 可以直接置 0，对于输入的 `UART_RTS` 可以不进行判断。串口接收到的数据通过 8 比特的 `rx_data` 送给上层模块。接收端与上层模块通过 `rx_finish` 和 `rx_ready` 两个信号来进行握手。

### 12.4.3 UART 调试功能设计

我们的 UART 调试模块通过调用串口发送模块及接收模块来进行串口通信，根据串口的输入确定需要执行的调试动作，并对 CPU 进行控制。示例调试程序提供了基本的 CPU reset、指令存储器加载和数据存储器加载功能。同学们可以根据自己的需求来对调试模块进行改造，增加更多功能。例如，单步调试、显示 PC，显示指令和数据存储器中的数据、显示寄存器内容等等。

示例 UART 调试工具的接口如下：

```

1 module uart_port(input clk,
2                   input clk100m,
3                   output rxd_out,
4                   input txd_in,
5                   output reg [31:0] dbgdata,
6                   input rst,
7                   output [31:0] instaddr,
8                   output reg[31:0] instdata,
9                   output reg iwe,
10                  output [31:0] dataaddr,
11                  output [31:0] mdata,
12                  output reg dwe,
13                  output reg cpuhalt,
14                  output reg cpureset
15                );

```

其中包括调试模块自身时钟 `clk`（建议与 CPU 时钟保持一致）及提供给收发模块的时钟 `clk100m`。串口的收发数据线 `rxd_out` 及 `txd_in`。串口调试工具对外提供的 32 比特调试信息 `dbgdata` 可以直接连在七段数码管上，用于显示调试结果，例如 PC，寄存器，存储器内容等等。为了刷新指令存储器和数据存储器，调试工具还输出了指令存储器地址 `instaddr`，指令数据 `instdata` 和指令存储器写使能 `iwe`，以及数据存储器对应的地址、数据和写使能信号。为了实现对 CPU 的控制，调试模块还输出了 CPU 暂停 `cpuhalt` 和 CPU 重置 `cpureset` 信号。

**调试模块对 CPU 的控制：**我们可以通过对 CPU 输入进行简单改造来实现对 CPU 的控制。

```

1 //main CPU

```



```

2 rv32ip mycpu(.clock(clk&~uart_halt),
3             .reset(BTNC|uart_rst),
4             .imemaddr(iaddr), .imemdataout(idataout), .imemclk(iclk),
5             .dmemaddr(daddr), .dmemdataout(ddata), .dmemdatain(ddatain),
6             .dmemrdclk(drdclk), .dmemwrclk(dwrclk), .dmemop(dop), .dmemwe(dwe),
7             .dbgdata(cpudbgdata));

```

这里将 `uart_halt` 信号取反后与 CPU 时钟进行了与操作。这样，当 `uart_halt` 是高电平时，CPU 时钟始终是为 0 的，即 CPU 不再动作。因此，串口调试模块可以通过 `uart_halt` 来暂停 CPU 的动作，观察 CPU 各项信息，然后再将 `uart_halt` 置 0，让 CPU 继续执行。同样，CPU reset 的输入也从仅支持硬件按钮 BTNC 重启，修改为在 `uart_rst` 信号有效时也可以重启 CPU。

#### 调试模块对存储器的控制

同样的，调试模块也可以对指令存储器和控制存储器进行控制。例如，我们可以将指令存储器的接口进行如下改写：

```

1 instmem instructions(
2     .addra(uart_halt?uart_iaddr[17:2]:iaddr[17:2]),
3     .clka(uart_halt?~uart_clk:iclk),
4     .dina(uart_idata),
5     .douta(idataout),
6     .ena(1'b1),
7     .wea(uart_iwe&uart_halt)
8 );

```

在 `uart_halt` 信号有效时，CPU 暂停工作。同时，指令存储器的地址、时钟、输入数据和写使能均由 UART 调试模块来进行控制。这样，UART 调试模块可以在 CPU 停下时对指令存储器进行任意的读写操作。这里需要注意的是存储器的时序问题，提供给存储器的时钟可能会需要与 UART 自身时钟错开半个周期，以保证在数据有效时（UART 时钟周期中间点）进行存储器的数据读写。UART 调试器对数据存储器的操作也类似，这里不再详述。

**调试模块的串口收发功能：**调试模块的串口收发功能通过两个先入先出的队列，`tx_buf` 和 `rx_buf`，来缓存串口通信数据。对于接收队列，串口收发时仅负责向队尾添加数据，递增 `rx_tail` 尾指针。而对于发送队列，该部分代码负责在队列中有数据时取出 `tx_head` 进行数据发送。具体对接收数据的

处理及控制发送数据是由 UART 调试器的状态机来负责实现的。由于 UART 调试器的时钟远高于串口的 115200Hz 时钟，所以在本实现中没有考虑收发缓冲溢出的问题。

```
1 // send and receive logic
2 always@(posedge clk)
3 begin
4     if(rst)
5     begin
6         tx_head<=4'd0;rx_tail<=4'd0;
7         tx_send<=1'b0; rx_gotdata<=1'b0;
8     end
9     else
10    begin
11        if(rx_ready)
12        begin
13            if(rx_gotdata==1'b0)
14            begin
15                rx_buf[rx_tail]<=rx_data;
16                rx_tail<=rx_tail+4'd1;
17                rx_gotdata<=1'b1;
18            end
19        end
20        else
21        begin
22            rx_gotdata<=1'b0;
23        end
24        if(tx_head!=tx_tail)//has data to send
25        begin
26            if(~tx_send&tx_ready) //tx free
27            begin
28                tx_data <= tx_buf[tx_head];
29                tx_head<= tx_head+4'd1;
30                tx_send<=1'b1;
31            end
32            else
33            begin
```

```
34         if(~tx_ready) tx_send<=1'b0;
35     end
36 end
37 else
38     begin
39         if(~tx_ready)
40             begin
41                 tx_send<=1'b0;
42             end
43     end
44
45 end
46 end
```



注意 Verilog 中的缓冲区不是 C 语言中的数组，建议不要在同一个时钟周期内对同一个缓冲区的不同位置进行多次读写操作，这样有可能对存储的读写时序要求太高，在综合时无法通过 RAM 实现，最终造成无法综合或出错。例如，当要通过串口输出多个字符时，不可在一个周期将多个字符同时写入缓存区，而是要分多个周期每次写入一个字符。由于这样实现比较复杂，建议同学们可以用七段数码管来实现调试信息的输出，而非使用串口直接输出。

**调试模块的状态机：**UART 调试模块的具体调试功能通过状态机来实现。在初始状态下，调试器等待用户输入的单字符命令，例如，'i' 表示加载指令存储器，'d' 表示加载数据存储器，'r' 表示重启 CPU 等等。当串口输入数据能够匹配对应指令，调试器将转移到具体的调试状态。例如，在匹配'r'之后，调试器将输出 `cpurest` 置 1，同时进入重启延迟状态，即维持 `reset` 信号一段时间后，再清除 `reset` 信号，并返回初始状态。对于加载指令和数据存储器，在进入加载状态后，调试器将首先暂停 CPU 的执行，同时，等待串口传输的 `.hex` 文件。我们之前介绍过存储初始化 `hex` 文件的格式，其中以 '@' 开头的行，后续为十六进制的地址，因此可以用状态机来解析 `hex` 文件中指示的加载地址，并将 UART 调试器的 `instaddr` 设置成对应的地址。`hex` 中数据部分每行表示存储器一个单元的数据，也是以十六进制表示，状态机可以将其转换后，通过控制存储器的写入数据端口和写使能端口，每收到一个数据就写入对应存储器，并将地址加一。当传输完毕后，用户可以输入 'q' 来结束加载，同时 `reset CPU`

并开始用新的指令或数据进行执行。

**思考：**如何实现其他调试功能？

- 如果要想实现单步执行，如何通过 `cpuhalt` 信号来控制 CPU 只执行一个周期即停止等待？
- 如果将 CPU 的指令地址送给 UART 调试模块，是否可以实现设置断点功能？
- 如何实现查看指令和数据存储器中给定地址内容的功能？
- 如何实现查看 CPU 中寄存器内容的功能？需要对 CPU 和 UART 调试器进行哪些改造？
- 在 Linux 系统下，是否可以通过 `makefile` 脚本直接进行串口加载代码操作？

## 12.5 实验验收要求

### 12.5.1 上板验收

本实验以分组方式进行，只需要进行上板验收，实验验收时需要演示完整的计算机系统。其中必须要实现的功能包括：

1. 接受键盘输入并回显在屏幕上
2. 支持换行、删除、滚屏等操作
3. 命令分析：根据键盘输入命令执行对应的子程序，并将执行结果输出到屏幕上。
  - 打入 `hello`，显示 `Hello World!`
  - 打入 `time`，显示时间
  - 打入 `fib n`，计算斐波那契数列并显示结果
  - 打入未知命令，输出 `Unknown Command`。

以上的计算机系统已经具有二十世纪八十年代末个人计算机的基本能力了，这样的系统可以完成很多有趣的功能。感兴趣的同学还可以自行思考，实现其他附加功能，附加功能可以考虑但不限于以下内容。

**硬件可选实现的功能：**

1. 扩展 CPU 支持的指令类型，可以支持系统调用、乘除法或自定义指令
2. 支持简单的中断机制
3. 增加外设种类，支持对板载 LED、七段显示等控制

4. 增加显示器复杂度，提供简化的图形界面
5. 使用 AXI Lite 总线来连接各类外设，将外设模块化，增加 DDR、串口，SD 卡等类型的外设

**软件可选实现的功能：**

1. 输入简单表达式，如  $(9+6*6)-3$ ，输出结果
2. 实现简单的 C 语言库函数，移植其他课程，例如拔尖班 PA 中的 AM 的代码<https://nju-projectn.github.io/ics-pa-gitbook/ics2021/2.3.html>
3. 编译运行流行的 benchmark，对 CPU 进行性能评估